What makes a sketch-based long-read mapper cheaper: nine changes to shmap, by the layer they act on

Dobromir Panchev Pesho Ivanov

Abstract

Summary: shmap-rs is a Rust port of *map-shmap* [1] that is 2.07–2.95× faster single-threaded and uses 7.1–7.3× less peak memory while producing byte-identical output. This note accounts for where that came from. 9 deliberate changes are responsible, together with the rewrite that carries them — and the ablation below shows the rewrite is the larger of the two. All 9 preserve output exactly, and each is compared against the upstream source it replaces at a pinned revision. They are presented in the order they had to be discovered: the cost structure the algorithm implies, the data structure that structure argues for, the algorithmic identities that let work be skipped, the parallel decomposition, and only last the code. That order is the result rather than the presentation: the largest wins are in the first two layers, and instruction-level work is worth least here, and is where four further attempts measured negative.

Availability: original `https://github.com/pesho-ivanov/shmap`; this port `https://github.com/d0bromir/shmap-rs`, MPL-2.0, version 2.0.0.

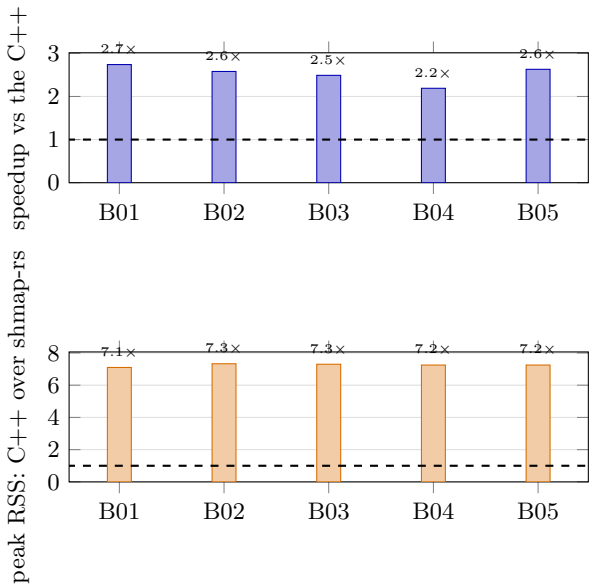


Figure 1: The two headline results, per benchmark, single-threaded on both sides: wall-clock speedup over the C++ (top) and how many times less peak memory the port uses (bottom). The dashed line is parity.

1 The theoretical base

map-shmap [1] is Ivanov and Medvedev's, and nothing below changes what it computes. It sketches read and reference by FracMinHash, attributes each seed match to the overlapping fixed-width buckets whose window contains it, prunes buckets with an admissible seed heuristic, and refines the survivors with a sliding window. Its per-read cost has three factors — reads, matches examined, and a logarithmic term in reference size — and the important property for an implementer is which of them is paid in *arithmetic* and which in *memory traffic*.

Almost all of it is memory traffic: seeds are looked up in an index far larger than the last-level cache, the buckets they touch are scattered across an array sized by the reference, and the refinement sweep walks a window last touched by a different read. Profiling agrees: `map::match_rest` is the costliest stage at 20.8% of CPU on average, and pointer-chasing rather than arithmetic. Two consequences order everything else. A change that reduces *how much memory is touched* outranks one that reduces instructions issued, because the machine is waiting either way; and the sizes that matter are those set by the *reference*, since a whole-genome run cannot escape them — which points at the accumulator.

2 The data structure

The C++ allocates one dense bucket array per reference segment, sized so that every window the algorithm could ever use has a slot (`buckets.h`, Table 1). Its width is a compile-time minimum rather than the width any particular read will use, so a human genome costs about fifteen gigabytes before a single read is mapped, whether one read is mapped or millions.

The observation that removes it is that *a read can only touch buckets within its own span*. The array does not have to be indexed by reference bucket; it has to be indexed by the buckets this read can reach, which is set by the read's own half-length. shmap-rs sizes it that way and keeps a sparse map as the fallback, and the two paths are tested to agree exactly. The footprint stops being a property of the genome and becomes a property of the query: 2.57–2.67 GB single-threaded against 18.85 GB, the lower panel of Figure 1.

One change, in one data structure, and it is most of the memory result. It also removed a contention bottleneck that had made multithreading nearly useless: a per-worker copy of a genome-sized array does not scale, and no thread tuning would have rescued it. The data structure had to be right before the parallel decomposition could be.

3 The algorithmic identities

Three changes skip work by proving it need not be done. None is an approximation; each is an identity, and the byte-identical output is the check.

The second-best search repeats the first. Computing mapq needs the second-best mapping, and the C++ searches for it with a second full pass over the same surviving buckets (`shmap.h`, Table 1). On the two

#	Change	C++ replaced	Effect, as measured when it landed	Exact
data structure				
1	Dense bucket array sized by the read's own half-length, not the reference	`buckets.h:76-97`	~4 MB/worker vs the C++'s constant ~15 GB — the 6.9-7.4x-less-memory headline (RESULTS.md §1) is almost entirely this. On a 6,000-read whole-genome run: `bucket_merge` 1342.1 s → 39.1 s, whole run 1995.6 s → 725.6 s	yes
algorithm				
2	Streaming multi-hit seeds	`shmap.h:105-140`	Not isolated as its own standalone percentage in the record; replaces ~O(hits) hashmap inserts with O(1) integer work per hit (§2)	yes
3	`RefineCache`	`shmap.h:508-520`	44% of `find_best_mapping` calls eliminated, flat across 1x/3x/10x coverage: `match_rest_for_best2` **-66 to -67%**, `refine` **-39 to -41%**, `query_mapping` **-9 to -11%** (§3)	yes
9	Final bucket ordering: an unstable sort on a packed key reproduces stable-sort output	`buckets.h:151-174`	Sorts 8-byte packed keys instead of 32-byte records — no isolated standalone percentage in the record, but real: moving 4x fewer bytes and skipping a stable sort's temporary allocation	yes
parallelism				
4	Multithreaded read mapping	— *none*	RESULTS.md §2 (single-thread and `-@4` per benchmark) and §3 (the full thread-scaling table, including the NUMA ceiling Q4/Q5 diagnosed)	yes
5	Hash-sharded index storage + chunked reference sketching, both parallel across `-@`	— *none*	`index_initializing` **8.4 s → 1.1-1.8 s**; chunked sketching **~18% off 'indexing'** at `-@8` (§5)	yes
6	Two-pass parallel FASTA parsing	— *none*	`index_reading` **4.4 s → 1.5-1.7 s** (§6)	yes
code				
7	Sketching/index hot loop	`sketch.h:461-463`	**~13-17% off 'index_sketching'** is the bounds-check-free iteration alone (§7); the other three items in this row have no isolated standalone percentage in the record	yes
8	Lower allocation/memory traffic	`types.h:43-71`	**~11-13% off peak RSS** is the bounded read-ahead alone; **~5% wall** is `lto="fat"` alone (§8); the rest are allocation-*count* reductions (e.g. ~300 M fewer heap allocations for single-occurrence seeds) without an isolated standalone percentage in the record	yes

Table 1: Every change that makes shmap-rs cheaper than the C++, grouped by the layer it acts on, with the upstream source each replaces at commit `63f1103`. Effects are what the change was worth against the build it landed on and are deliberately not restated against the current one. They do not sum: the whole-run comparison is measured separately. *Exact* means the mapping output is byte-identical.

fixed-length metrics, scoring a bucket turns out to be a pure function of its *location*: it never reads the mutable state that pruning updates between the passes, and it restores its own scratch exactly, so the second pass can replay the first's scores. It is deliberately not applied to the two bucket-based metrics, where the score is built out of exactly that mutable state. Because the acceptance threshold ratchets upward during the first pass, a bucket discarded late there can still clear the second pass's flatter cutoff — so the win is partial by construction, which is why it is roughly two calls in five and not all of them.

A repetitive seed does not need a hash map. The C++ builds a fresh scratch hash map per seed to accumulate its hits; the hits are already sorted, so the map is redundant and the same buckets can be accumulated in place at constant integer cost per hit.

An unstable sort can reproduce a stable one. Ordering the final buckets by descending match count is the original paper's own Algorithm 4 "Optimization 2". Packing the tie-break into the sort key makes an unstable sort reproduce exactly what a stable sort would give, over keys a quarter the size of the records. Here the port is stronger than the source rather than merely faster: the C++'s `std::sort` guarantees no particular order among equal keys, even between its own runs, and this does.

4 The parallel decomposition

3 of the 9 exist only because the C++ is single-threaded, so they replace nothing upstream: threaded read mapping, hash-sharded index storage with chunked reference sketching, and a two-pass count-then-fill FASTA reader. The constraint that shapes all three is that output must not depend on thread count, which rules out any decomposition whose result depends on completion order: a sketch window is a pure function of the bases under it, a shard's contents are set by the hash rather than by which worker filled it, and the FASTA reader's second pass writes what its first counted. Determinism is checked rather than assumed, on every thread count and benchmark (15 of 15 checks).

5 The code

Only two of the 9 are instruction-level, and they are last because they are worth least. The sketching hot loop precomputes its rotation tables, iterates without bounds checks, hashes once per lookup instead of twice, and sizes its output buffer from the actual distribution. The allocation work stores a single query position inline rather than in a one-element vector, has matches borrow their seeds instead of copying them, indexes the difference histogram densely, and discards the reference sequence after sketching, where the C++ keeps a second full copy of the genome for the whole run.

Their effects are in Table 1: real, but small beside the first two layers. The stronger evidence for the ordering is negative — hand-written SIMD sketching, two-bit packing, software prefetching of the pruning lookups, and per-node index replication implemented five ways, were each built or probed at full scale and each measured a loss or a wash. The sketching loop is load-port bound at six level-one loads per base, so more lanes add pressure where there is none to spare, and packing re-

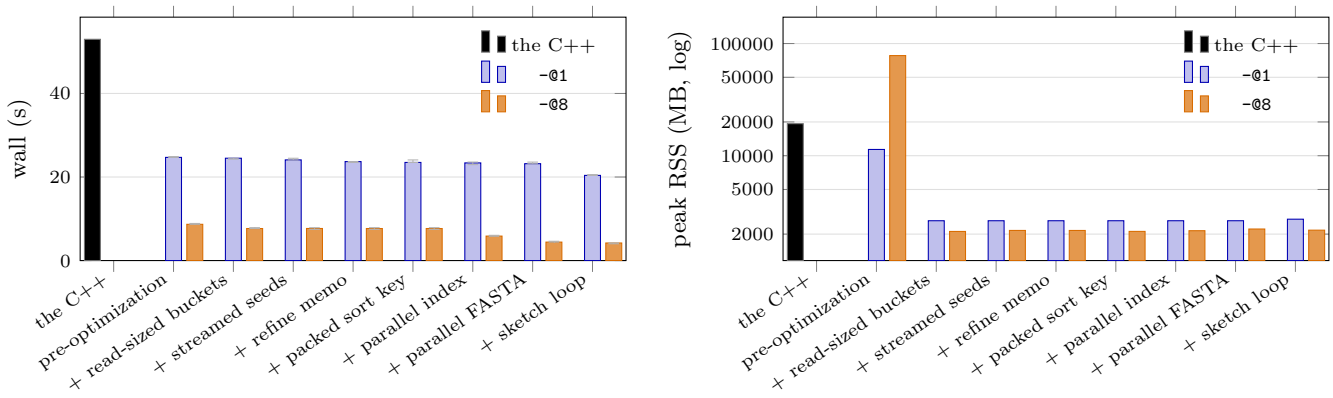


Figure 2: Every optimization, put back one at a time, on the whole genome (D6-ONT24K, REF-HS1). The leftmost bar is the C++ itself; *pre-optimization* is this port with every ablatable change switched off (`SHMAP_ABLATE`), and each rung to the right switches exactly one more back on, so adjacent rungs differ by one change and nothing else. End to end the C++ is 2.64× the shipped mapper here, of which 2.15× is the port itself and 1.23× the ladder; the product returns the first. The two series are the two thread counts: their gap is row 4, row 8 is not ablatable, and the C++ has no `-@N` counterpart. Left: wall clock, median of 5 round-robin runs of the whole ladder, whiskers at min–max. Right: peak resident set, log scale; the accumulator the first change removes is sized by the reference and allocated *per worker*, which is why the `-@8` baseline stands above even the C++. Every rung's mapping is byte-identical; the run fails otherwise. Numbers, per-stage timers and the decomposition in `figure_ablation.tsv` and `reconcile.tsv`.

moves the wrong two of those loads. When a workload waits on memory, instruction-level effort is the layer with least left in it.

6 What each one is worth

Table 1's effects were each measured against the build that change landed on, months apart and on different inputs; they share no baseline, machine or read set and do not sum, so the table cannot say how much of the result is which change. Figure 2 answers that, on the whole genome on galaxy, which has 128 cores and the memory to hold the pre-optimization accumulator once per worker — the reason the figure could not be produced on a smaller machine. End to end the C++ is 2.64× the shipped mapper there and uses 7.49× the peak memory: Figure 1's headline, reproduced inside the ladder's own protocol.

Every change is switchable at run time, in one instrumented build, back to the code path it replaced — one binary, one machine, one compiler, one input. That is what makes a step attributable, and attributable to that change *given the ones to its left*: what the parallel index is worth depends on whether each worker still carries a genome-sized accumulator. Every bar's mapping is compared byte for byte against the leftmost, a stricter test of the exactness claim than the suite's. Those switches are not in the released mapper — a branch that exists only to be measured is paid for by everyone who runs the program — so they live on an archived, never-merged branch, and the harness refuses a build that lacks them. 2 of the 9 cannot be a rung and are reported as absent rather than estimated: one is a capability rather than a branch, the other type-level, as below.

The step into the ladder is the largest, and it is not one of the nine. Getting from the C++ to *pre-optimization* — the port itself, before any ablatable change is switched back on — is 2.15×, against 1.23× for the whole ladder that follows it; the two compose by multiplication and return the 2.64× measured directly between the ends, which is what makes the decomposition checkable. Two things sit in it: the allocation-and-layout row, which is type-level — matches borrow their seeds, the single-occurrence case is stored inline — so reversing it is a different program rather than a different branch, which is why it cannot be a rung; and the rewrite itself, a different language, allocator and standard library. This note does not separate them.

Memory is where the layering is starkest. The 7.49× splits 1.70× for the port against 4.42× for the ladder, and almost all of the second is one bar: the accumulator the C++ sizes by the reference, a slot per 5 bases of it. That is why the C++'s own footprint barely moves between benchmarks — 6 MB across read sets differing by more than an order of magnitude in size — and why the ratio *grows* with worker count, from 4.19× to 36.00× at 8: the array is per worker, so what was removed is multiplied by the thread count while what remains is not.

7 What none of them changed

All 9 preserve output exactly, enforced rather than claimed: the mapping is byte-identical across every thread count and against the C++, ground-truth accuracy is unchanged at 98.76–99.21%, and any drop in mapped or confident reads blocks a merge.

References

[1] Ivanov,P. and Medvedev,P. map-shmap: practical long-read mapping with a seed heuristic on sketches. Manuscript in preparation.